Lead Enrichment Platform

Architecture and delivery plan for a 50 to 100 million row internal lead generation tool

Prepared 2 November 2026. Status: proposal for review.

1. Summary

> **Recommendation:** keep Next.js for the product, move all data and job state into **managed PostgreSQL 17/18**, run enrichment on **long-running worker containers** that pull work in **batches straight from Postgres** (`FOR UPDATE SKIP LOCKED`), use **Valkey** only for provider rate limits, and add **ClickHouse** as a read model once tables grow past a few million rows and people need fast filter, sort and export on any column.

- **The speed problem is mostly the work model, not the database.** One HTTP request or one queue message per cell, sent through a hosted backend API, caps out quickly. Batching 500 to 1,000 cells per unit of work and writing results back in bulk gives 10x to 50x more throughput on the same budget.
- **Column dependencies become pull-based.** A worker asks "which rows have column B done and column C still pending?". No fan-out of billions of queue messages, no lost jobs, automatic resume after crashes.
- **No dependency on a single proprietary queue vendor.** The queue is a Postgres table. Valkey (Linux Foundation, BSD licence) is a drop-in for Redis and is used only for counters, so it can be swapped or removed.
- **Target throughput:** 30,000+ provider calls per minute (10x the current 3,000), limited by provider quotas and budget rather than our stack.

2. The problem, restated

A user uploads a file or sets a source filter (for example *gyms in Mumbai*). The system pulls matching records from a source API and stores them as rows in a table. Each further column is an enrichment step whose input is an earlier column:

A: Company	B: Decision maker	C: Email	D: ...
Entrepreneurial Labs	Shekhar Gupta (linkedin.com/in/...)	shekhar@...	phone, score, ...
input from source or file	needs A	needs B	needs any earlier column

So every table is a small **DAG of columns**, executed for every row, where each column calls one or more third-party providers. The platform must handle **50 to 100 million rows** in total.

Assumptions (please correct)

- 100M rows is the total across all tables; a single table is usually 1k to 5M rows.
- An average table has 3 to 6 enrichment columns, so roughly 300M to 600M provider calls over the dataset's life.

- Tens of internal users, not thousands. Reads are a spreadsheet-style grid plus CSV export.
- Provider rate limits and per-call cost are the real ceiling; our system must never be the bottleneck.

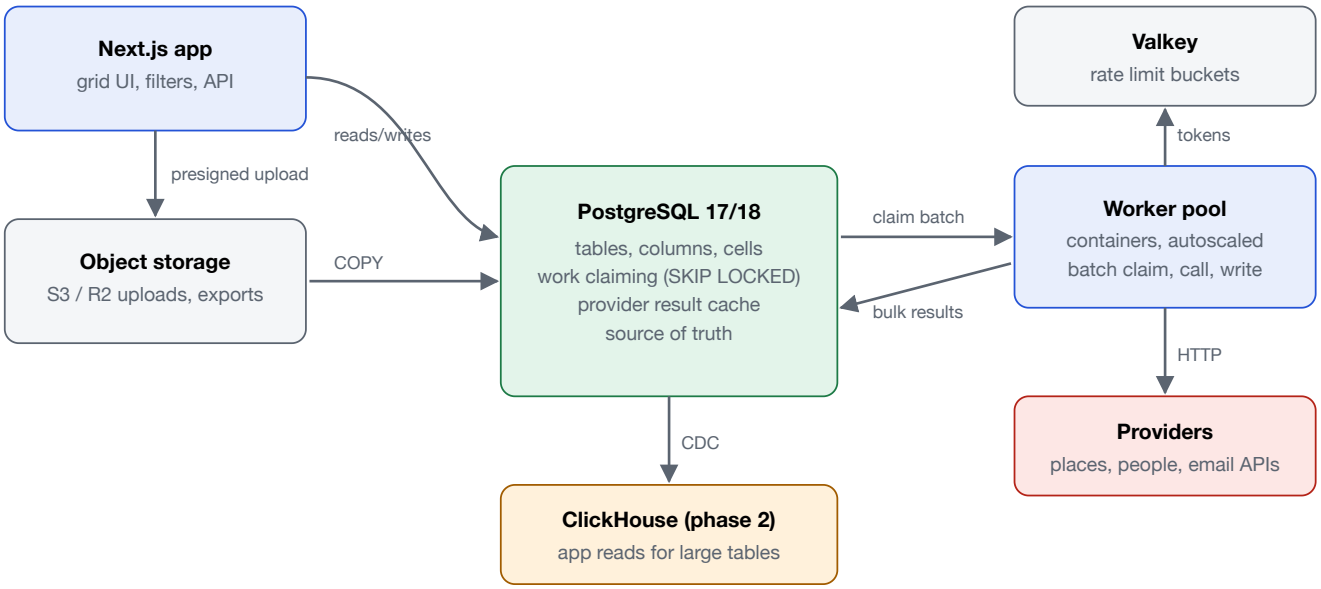
3. Why the current stack is slow

Cause	Effect
Per-cell jobs (one job, one request, one write per cell)	Overhead per cell is larger than the provider call itself; queue and DB become the bottleneck at a few thousand per minute.
Database reached through a hosted no-code API (Xano)	Every read and write is an HTTP round trip with platform rate limits; no bulk `COPY`, no batched upserts, no control over indexes or partitions.
Dependencies pushed as follow-up jobs	A failed or dropped message silently leaves a row stuck; retries re-run whole chains.
No shared result cache	The same company or person is enriched again for every upload, wasting time and money.
Queue state in a separate Redis-based service	Two sources of truth, and a vendor change forced this rewrite.

4. Numbers that drive the design

Quantity	Estimate	Consequence
Provider calls for 100M rows x 3 columns	300M	At 3,000/min this is **69 days**. At 30,000/min it is **7 days**. With a 40% cache hit rate, about 4 days.
Cell writes at 30,000/min	500/s	Trivial for Postgres when batched (one `UPDATE ... FROM (VALUES ...)` per 500 cells = 1 statement/s per worker).
Cells stored (100M rows x ~5 columns)	500M	~120 to 200 GB including indexes. One well-sized Postgres instance; no distributed database needed.
Concurrent outbound HTTP	500 to 2,000 in flight	I/O bound. 4 to 8 small worker containers on Node/Bun or Go handle this.
Grid reads	page of 100 rows, any filter/sort	Fine in Postgres up to a few million rows per table; above that, serve from ClickHouse.

5. Recommended architecture



Components

Layer	Choice	Why
Product UI and API	Next.js (keep)	No reason to change; the team knows it. Grid with a virtualised data grid (Glide Data Grid or AG Grid) and server-side paging.
Primary database	Managed PostgreSQL 17/18 (AWS RDS/Aurora, PlanetScale Postgres, Neon or Supabase)	Handles 500M cells on one node, real transactions for work claiming, `COPY` for fast import, mature tooling, no lock-in. Pick a plan with provisioned IOPS for sustained writes.
Job queue	Postgres itself (cells table + `SKIP LOCKED`); `pg-boss` for coarse jobs (parse upload, source fetch, export)	Queue state and data live in one place, so they can never drift. Battle-tested pattern, no extra service.
Workers	TypeScript on Bun (or Go) in long-running containers (ECS Fargate, Fly.io or Railway)	Serverless functions are a poor fit for sustained high-concurrency HTTP. Containers keep connections warm and scale on backlog.
Rate limiting	Valkey (ElastiCache Valkey, Upstash or self-hosted)	Atomic token buckets shared by all workers per provider and API key. Open source Redis fork, Redis protocol compatible.
Files	S3 or Cloudflare R2, parsed with DuckDB	Browser uploads straight to storage; DuckDB reads CSV/XLSX/Parquet of millions of rows in seconds and streams into Postgres `COPY`.
Analytics read model	ClickHouse Cloud, fed by CDC (ClickPipes/PeerDB or Sequin)	Sub-second filter, sort, group and export on any column across 100M rows. Added only when Postgres reads get slow.

Live updates	Server-sent events from Next.js, polling `updated_at` deltas every 1 to 2 s	Simple, no extra realtime service; cells light up as workers finish batches.

6. Data model

Narrow cells (one row per row x column) keep updates cheap: finishing an email cell rewrites ~200 bytes, not the whole row. Hash partitioning by table keeps each partition small enough for fast vacuum and index maintenance.

```
create table lead_tables (
  id          text primary key,
  workspace_id text not null,
  name        text not null,
  source      jsonb not null,           -- {"kind":"file"} or
{"kind":"places","query":"gyms","city":"Mumbai"}
  row_count   bigint not null default 0,
  created_at  timestamptz not null default now()
);

create table lead_columns (
  id          text primary key,
  table_id    text not null references lead_tables,
  position    int not null,
  name        text not null,
  kind        text not null,            -- input | enrich | formula
  depends_on  text[] not null default '{}',   -- column ids this column reads
  config      jsonb not null            -- providers in waterfall order, field mapping, run
condition
);

create table cells (
  table_id    text not null,
  row_id      bigint not null,
  column_id   text not null,
  status      smallint not null,        -- 0 waiting, 1 ready, 2 running, 3 done, 4 empty, 5
failed, 6 skipped
  value       jsonb,
  lease_until timestamptz,
  attempts    smallint not null default 0,
  error       text,
  updated_at  timestamptz not null default now(),
  primary key (table_id, column_id, row_id)
) partition by hash (table_id);       -- 32 partitions

create index cells_ready on cells (table_id, column_id, row_id) where status = 1;
create index cells_changed on cells (table_id, updated_at);

create table provider_cache (
  provider    text not null,
  input_hash  bytea not null,           -- sha256 of normalised input (domain, linkedin url,
name+company)
  response    jsonb not null,
  fetched_at  timestamptz not null,
  primary key (provider, input_hash)
);
```

Rows themselves need no table: a row is its `row_id` plus its input cells. Typed entity tables (`companies` by domain, `people` by LinkedIn URL) can be added later for cross-table dedupe and search.

7. The dependency engine

States and promotion

1. When a table is created, every enrichment cell is inserted as `waiting` in one bulk statement.
2. When a batch of cells in column B reaches `done`, the same transaction promotes the matching cells of every column that depends on B from `waiting` to `ready`, if all of their other dependencies are also done. If B ended `empty` or `failed`, dependents become `skipped`.
3. Workers only ever look at `ready` cells, using the small partial index.

Claiming a batch

```
with batch as (
  select table_id, column_id, row_id from cells
  where table_id = $1 and column_id = $2 and status = 1
  order by row_id
  limit 500
  for update skip locked
)
update cells c set status = 2, lease_until = now() + interval '5 minutes', attempts = attempts
+ 1
from batch b
where c.table_id = b.table_id and c.column_id = b.column_id and c.row_id = b.row_id
returning c.row_id;
```

- Many workers run this at once; `SKIP LOCKED` guarantees each cell goes to exactly one worker.
- A reaper resets `running` cells whose lease expired back to `ready`, so a crashed worker loses nothing.
- A scheduler picks which (table, column) to claim next with weighted fair sharing, so one 5M-row table does not starve everyone else.

Processing a batch

1. Load the input cells for the 500 rows in one query.
2. Normalise inputs and look them up in `provider_cache` in one query. Hits are written straight away.
3. For misses, take tokens from the provider's Valkey bucket and call the provider with bounded concurrency (use the provider's bulk endpoint when it has one).
4. **Waterfall:** rows a provider could not answer move to the next provider in the column's list in the same pass.
5. Write all results, cache entries and dependent promotions in one transaction.
6. Retry transient errors (429, 5xx, timeouts) with backoff up to 3 attempts; mark permanent errors `failed` with a reason the user can see.

User actions this supports for free

- Run one column, run all, run only the first N rows (cost preview), pause or cancel a table.
- Re-run failed cells or change a column's config and re-run: reset its cells to `ready`; dependents reset to `waiting`.
- Per-cell status, error message and provider shown in the grid.

8. Rate limits, caching and cost control

Concern	Mechanism
Provider quotas	One token bucket per (provider, API key) in Valkey, refilled at the contracted rate. Multiple keys per provider are pooled. Workers shrink concurrency automatically on 429s.
Duplicate spend	`provider_cache` with a per-provider TTL (for example 90 days for emails, 30 for job titles). Also dedupe inside a batch before calling.
Budget	Estimated cost shown before a run (rows x providers x unit price minus expected cache hits); per-workspace daily credit cap enforced by the scheduler.
Fairness	Each table gets a share of each provider's bucket so concurrent users all see progress.
Observability	Per provider: calls/min, hit rate, error rate, p95 latency, spend. Per table: cells by status, ETA. OpenTelemetry to Grafana or Axiom.

9. Ingestion flows

File upload

1. Browser uploads directly to S3/R2 with a presigned URL (no file passes through Next.js).
2. A `pg-boss` job runs DuckDB over the file: detects columns, trims, normalises domains and URLs, removes duplicates.
3. Rows stream into Postgres with `COPY` (1M rows in well under a minute); enrichment cells are created as `waiting` / `ready`.

Source filter (for example gyms in Mumbai)

1. The filter is stored on the table as `source`.
2. A source job pages through the source API (places or company search), writing each page as a batch of rows, so the grid fills while it runs.
3. Large areas are split into tiles or sub-queries (by locality or pincode) to get past per-query result caps, then deduped by place id or domain.

10. Reading 100M rows in the grid

- **Phase 1 (Postgres only):** pages are fetched by `row_id` range and pivoted with one query on the primary key. Filters on a few common columns (status, has email, city) use expression indexes. This is fast for tables up to a few million rows.
- **Phase 2 (ClickHouse):** CDC streams `cells` into a `ReplacingMergeTree` keyed by (table_id, row_id) with a `Map` of column values. Arbitrary filter, sort, count and full-table CSV export become sub-second to a few seconds across 100M rows. Writes still only go to Postgres.
- Exports run as background jobs that write a file to S3/R2 and hand back a download link.

11. Options considered

Primary database

Option	Fit	Notes
PostgreSQL 17/18 (managed)	Recommended	Right size for 500M cells, transactional work claiming, `COPY`, JSONB, partitioning. Many hosts, easy to move between them.
Supabase / Neon	Good	Both are Postgres, so the same design applies. Good developer experience; check sustained-write performance and storage pricing at 200 GB+.
ClickHouse alone	Read side only	Excellent scans and filters, but frequent per-cell updates and job locking are not what it is built for.
CockroachDB / YugabyteDB / TiDB	Overkill	Distributed SQL helps beyond a few TB or multi-region writes. Higher latency and cost per write; not needed at 100M rows.
MongoDB / DynamoDB	Not chosen	Flexible rows, but weak ad-hoc filter and sort across columns, and DynamoDB scans get expensive. Job claiming is harder.
Xano (current)	Replace	HTTP-only access, platform limits, no bulk paths. Root of the slowness.

Queue and orchestration

Option	Fit	Notes
Postgres `SKIP LOCKED` + pg-boss	Recommended	Batches of cells as the unit of work, one source of truth, nothing new to run.
Hatchet / Trigger.dev / Inngest	Optional	Nice dashboards, retries and concurrency keys. Fine for coarse jobs; per-cell runs would be too many events. Hatchet is Postgres-backed and self-hostable.
Temporal	Overkill	Strong durability, but heavy to operate and per-row workflows create billions of history events.
BullMQ on Valkey	Possible	Fast, but splits queue state from data and recreates the old architecture.
Kafka / Redpanda / SQS	Not needed	Throughput of 500 cells/s does not need a log broker; adds ops and a second source of truth.

Redis replacement

Valkey is the Linux Foundation fork of Redis (BSD licence), supported by AWS, Google Cloud and others and protocol compatible, so any Redis client works. DragonflyDB and Microsoft Garnet are also compatible alternatives. In this design Valkey only holds rate limit counters, so losing it pauses calls but never loses data.

12. Hosting and rough monthly cost

Item	Size	Estimate (USD/month)

Postgres	8 vCPU / 32 GB, 500 GB provisioned storage, 1 replica	700 to 1,300
Workers	4 to 8 containers, 1 vCPU / 2 GB, autoscaled on backlog	100 to 300
Valkey	small managed instance	20 to 60
Object storage	uploads and exports	10 to 30
ClickHouse (phase 2)	small cloud service	250 to 600
Next.js hosting	existing	unchanged

Infrastructure is small next to provider spend: at even USD 0.002 per call, 300M calls cost USD 600,000, which is why the cache and cost preview matter most.

13. Delivery plan

Phase	Duration	Scope	Exit criteria
0. Spike	Week 1	Schema, claim query, one provider adapter, load test with 10M synthetic rows	Sustains 30k simulated calls/min with mock providers; p95 claim under 50 ms
1. Core engine	Weeks 2 to 4	Upload via S3 + DuckDB, source fetch, column DAG, waterfall, cache, Valkey limits, retries, reaper	Gyms in Mumbai to company, person, email end to end; crash a worker mid-run and nothing is lost
2. Product	Weeks 4 to 6	Grid with live updates, run controls, cost preview, CSV export, provider dashboard	Internal users run real tables without help
3. Migration	Weeks 6 to 7	Export Xano tables, bulk load, backfill cache from existing results, run both side by side then cut over	Row counts and sampled values match; Xano read-only
4. Scale reads	Weeks 7 to 9	ClickHouse via CDC, large-table filter/sort/export	Filter and sort on a 20M-row table under 1 s

14. Risks and open questions

Risks

- Provider quotas, not our system, will cap speed. Negotiate limits or pool keys early.
- Hot partitions if one huge table dominates; fair scheduling and hash partitioning mitigate it.
- Table bloat from frequent status updates: tune autovacuum per partition and keep cells narrow.
- Legal: scraping LinkedIn and storing personal data. Use compliant providers, record source per cell, and follow India's DPDP Act and GDPR for EU contacts.

Questions for the team

- Which providers are used today, and what are their contracted rate limits?
- Is 100M rows total or per table? Largest single table expected?
- How many users run tables at the same time?
- Preferred cloud (AWS, GCP, other) and monthly infra budget?
- What exactly was the Redis-based component that changed hands, and what did it do beyond queueing?
- How long must results be kept, and can cached data be shared across workspaces?